北师港浸大 HPC 简明使用手册

V2026313

请首次注册使用集群的同学，先用 ssh 命令行方式登录一次，用于自动创建家目录，后续可以用 web 方式或者 ssh 方式登录，以后再登录就可以选择 ssh 或 web 直接登录。

1. 命令行方式登录

通过 ssh 工具（putty/MobaXtrem）或者命令，登录服务器，常见方式为：

- 直接使用命令

ssh username@hpc.bnbu.edu.cn

密码：统一密码

输入密码即可登录成功，首次登陆成功后，后续即可使用 WEB 界面或 ssh 工具（putty/MobaXtrem）登录集群。

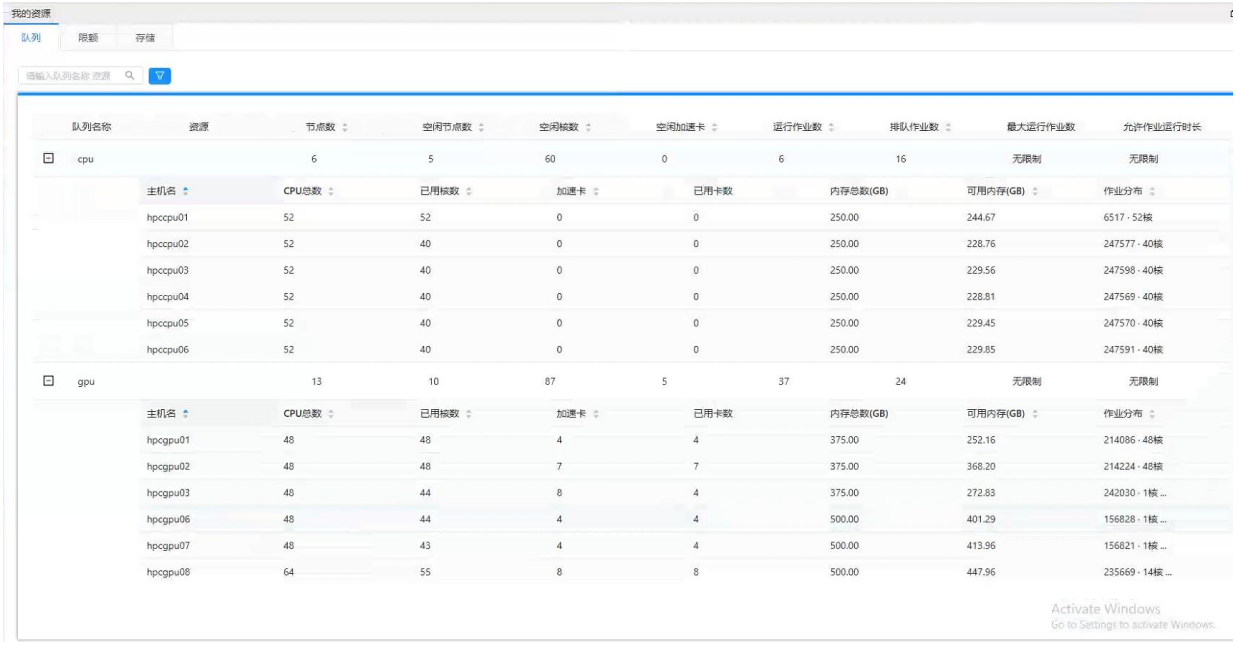
强烈不推荐使用 xshell，有木马，可能会导致服务器受攻击。

2. 使用 WEB 浏览器登录

使用新版 Chrome、Edge 或 Firefox 浏览器，访问 https://hpc.bnbu.edu.cn/，即可弹出登录用户名与密码对话框：

验证通过后，登录界面如下：

点击右上角“我的资源”，弹出菜单即可看到可用的资源类型以及目前已用和没有使用的：



我的资源

队列 限额 存储

请输入队列名称 资源

队列名称	资源	节点数	空闲节点数	空闲核数	空闲加速卡	运行作业数	排队作业数	最大运行作业数	允许作业运行时长
cpu		6	5	60	0	6	16	无限制	无限制
	主机名	CPU总数	已用核数	加速卡	已用卡数	内存总数(GB)	可用内存(GB)	作业分布	
	hpccpu01	52	52	0	0	250.00	244.67	6517 · 52核	
	hpccpu02	52	40	0	0	250.00	228.76	247577 · 40核	
	hpccpu03	52	40	0	0	250.00	229.56	247598 · 40核	
	hpccpu04	52	40	0	0	250.00	228.81	247569 · 40核	
	hpccpu05	52	40	0	0	250.00	229.45	247570 · 40核	
	hpccpu06	52	40	0	0	250.00	229.85	247591 · 40核	
gpu		13	10	87	5	37	24	无限制	无限制
	主机名	CPU总数	已用核数	加速卡	已用卡数	内存总数(GB)	可用内存(GB)	作业分布	
	hpcgpu01	48	48	4	4	375.00	252.16	214086 · 48核	
	hpcgpu02	48	48	7	7	375.00	368.20	214224 · 48核	
	hpcgpu03	48	44	8	4	375.00	272.83	242030 · 1核 ...	
	hpcgpu06	48	44	4	4	500.00	401.29	156828 · 1核 ...	
	hpcgpu07	48	43	4	4	500.00	413.96	156821 · 1核 ...	
	hpcgpu08	64	55	8	8	500.00	447.96	235669 · 14核 ...	

Activate Windows
Go to Settings to activate Windows.

命令行希望查询资源量以及使用量，则参照下命令：

sinfo -N -O "NodeList:15,CPUsState:20,Gres:20,GresUsed:30"

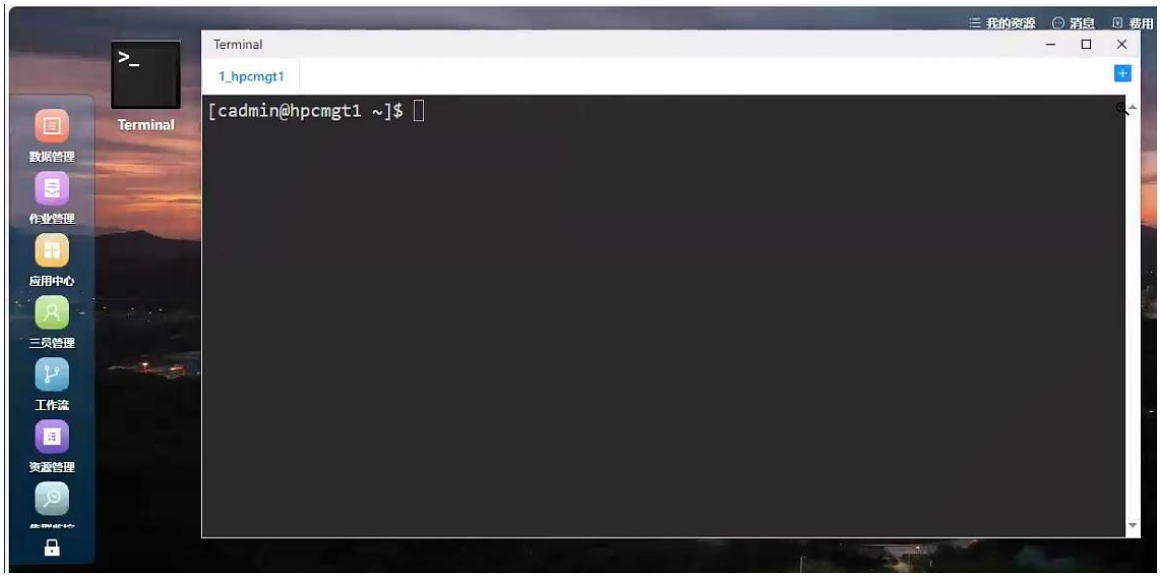
```
Last login: Mon Mar  9 15:29:04 CST 2026 on pts/53
(base) [redacted]hpcmgt ~]$ sinfo -N -O "NodeList:15,CPUsState:20,Gres:20,GresUsed:30"
NODELIST       CPUS(A/I/O/T)       GRES                GRES_USED
hpccpu01       52/0/0/52           (null)              gpu:0
hpccpu02       52/0/0/52           (null)              gpu:0
hpccpu03       52/0/0/52           (null)              gpu:0
hpccpu04       52/0/0/52           (null)              gpu:0
hpccpu05       52/0/0/52           (null)              gpu:0
hpccpu06       52/0/0/52           (null)              gpu:0
hpcgpu01       48/0/0/48           gpu:v100:4          gpu:v100:4(IDX:0-3)
hpcgpu02       48/0/0/48           gpu:v100:7          gpu:v100:7(IDX:0-6)
hpcgpu03       4/44/0/48           gpu:v100:8          gpu:v100:4(IDX:0-3)
hpcgpu06       4/44/0/48           gpu:a100:4          gpu:a100:4(IDX:0-3)
hpcgpu07       3/45/0/48           gpu:a100:4          gpu:a100:4(IDX:0-3)
hpcgpu08       14/50/0/64          gpu:4090:8          gpu:4090:7(IDX:0-5,7)
hpcgpu09       8/56/0/64           gpu:4090:8          gpu:4090:8(IDX:0-7)
hpcgpu10       13/51/0/64          gpu:4090:8          gpu:4090:8(IDX:0-7)
hpcgpu11       8/56/0/64           gpu:4090:8          gpu:4090:2(IDX:0,5)
hpcgpu12       64/0/0/64           gpu:4090:8          gpu:4090:8(IDX:0-7)
hpcgpu13       16/48/0/64          gpu:4090:8          gpu:4090:8(IDX:0-7)
hpcgpu14       9/55/0/64           gpu:4090:8          gpu:4090:6(IDX:0-5)
hpcgpu15       9/55/0/64           gpu:4090:8          gpu:4090:8(IDX:0-7)
(base) [s230026119@hpcmgt ~]$
```

最后一列，GRES_USED 就是已经使用的资源量：

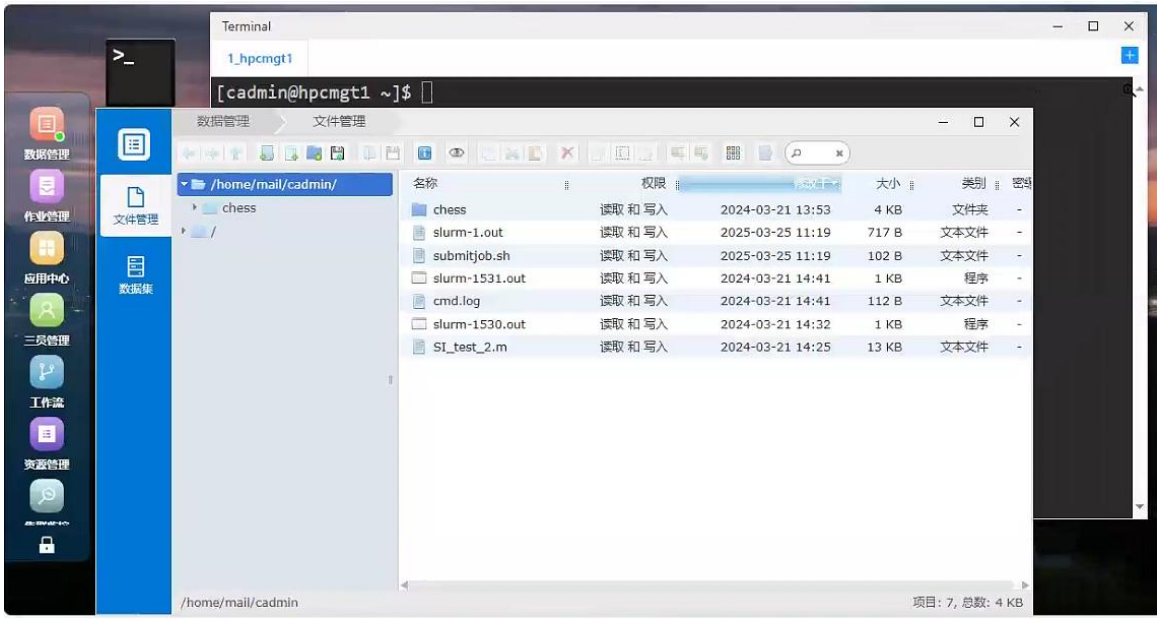
CPU 解释：如 52/0/0/52 表示总共 52 核（Total），0 核在使用（Allocated），0 核空闲(Idle)，非正常或特殊状态的核心数。（Other）。

GPU 解释：GRES 显示总数（如 4 张 A100），GRES_USED 显示已占用的编号（如 IDX:0 表示第一张被占用，而 IDX:0-3 表示 4 张卡在使用）。

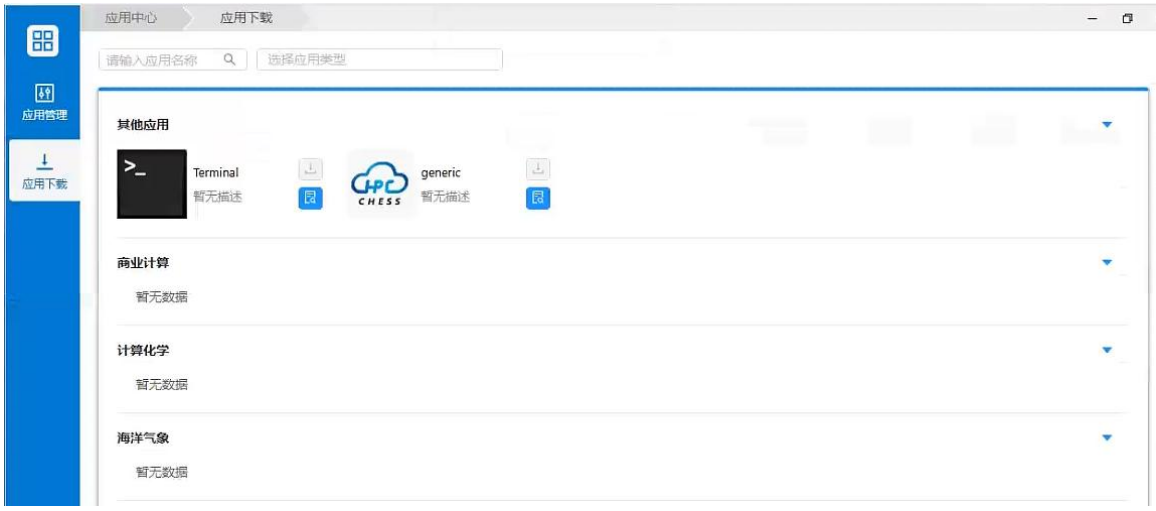
点击桌面图标“Terminal”，可直接使用 Linux 命令行，如下图



点击左侧“数据管理”可对个人数据进行上传、下载、删除、简单编辑等操作。



点击左侧“应用中心”打开应用中心选项卡，点击“应用下载”，可选择下载 generic 应用模板：



此时关闭该选项卡返回桌面，可见新的 generic 图标，点击打开，选择“提交”标签页，输入参数后可提交普通作业，如感觉可选参数不方便可通过“应用模板”进行管理修改。

因为现在服务器有两种不同的队列，分别是 CPU 队列，和 GPU 队列，执行 sinfo 可以看到：

```
[root@hpcmgt ~]# sinfo
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
cpu*         up   infinite      6   idle hpccpu[01-06]
gpu          up   infinite      3   idle hpcgpu[01,04-05]
[root@hpcmgt ~]#
```

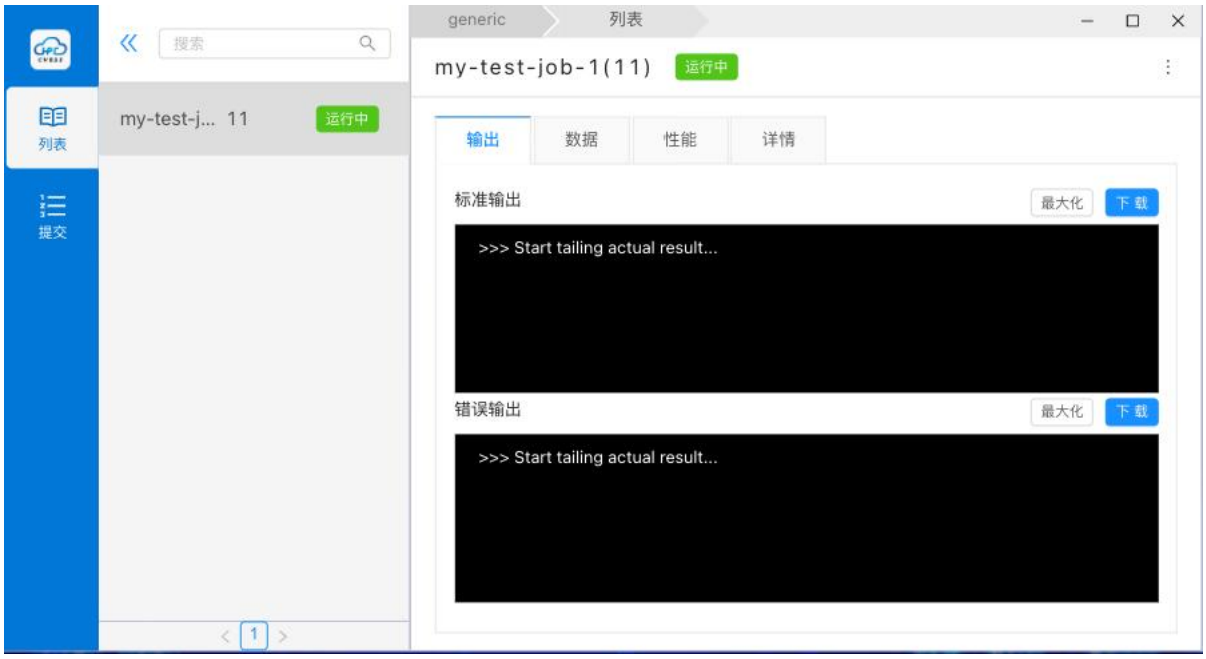
其中 CPU 队列是 6 台机器，每台 2 颗 Gold 5320 CPU，单机 52 个 CPU 核，内存 256GB；

GPU 队列目前是 3 台机器，每台机器有 4 块卡，其中 hpcgpu01 是单机 4 块 V100 GPU，hpcgpu04、hpcgpu05 是单机 4 块 A100；

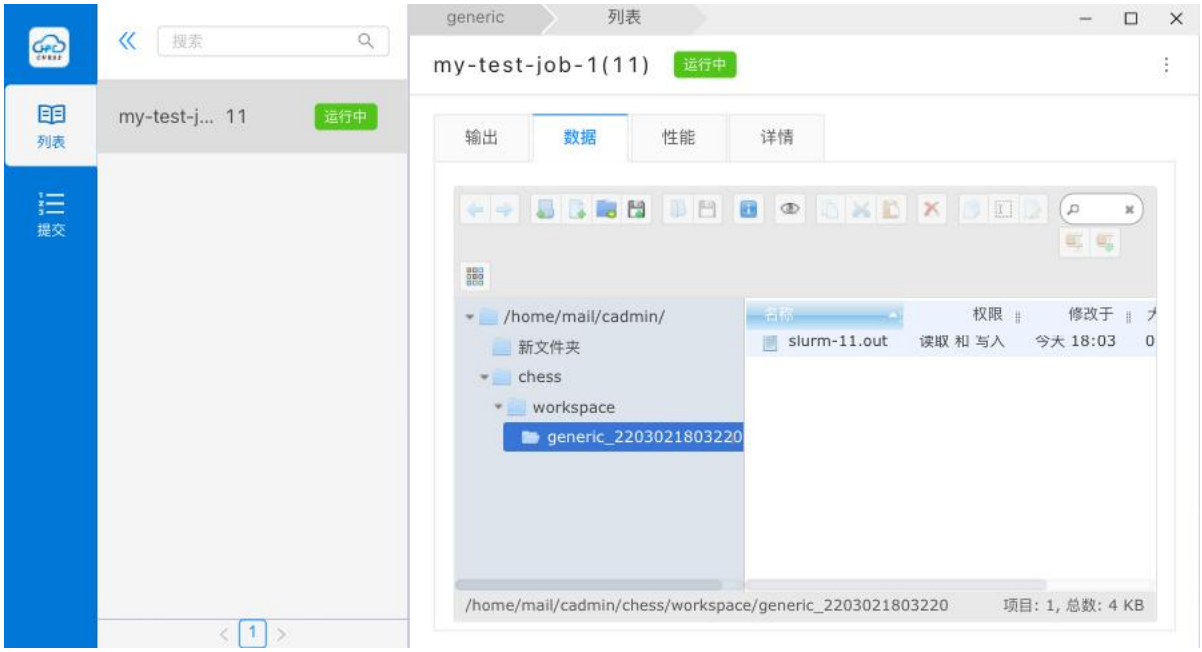
通过使用#SBATCH -p gpu 提交到 GPU 队列。

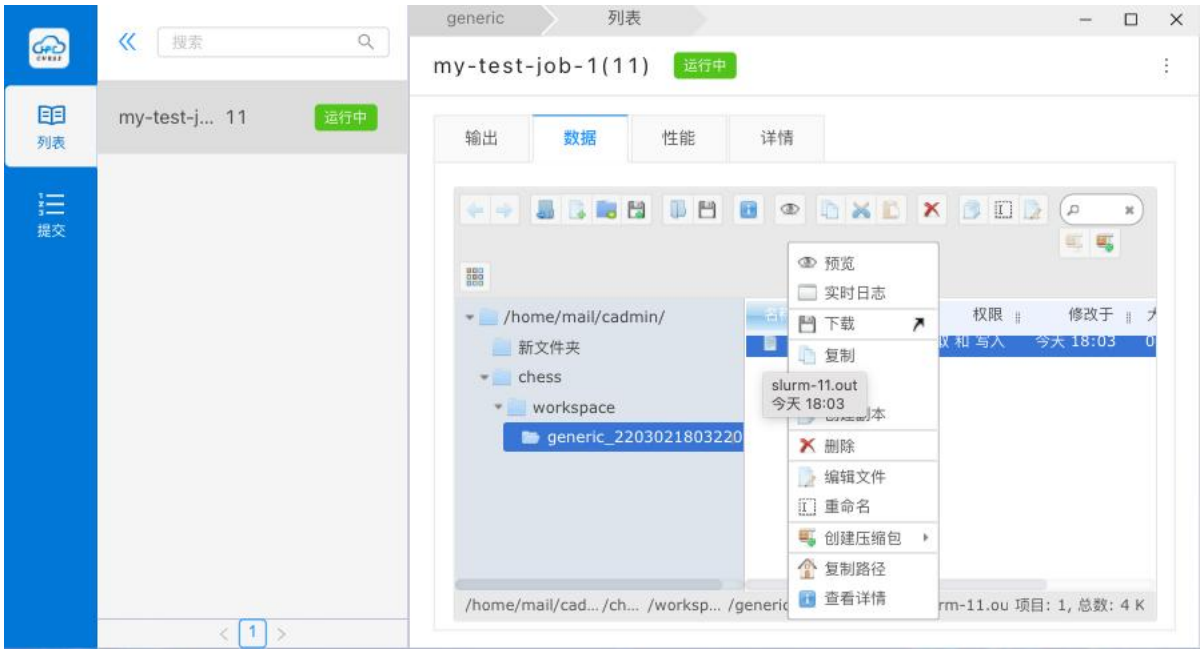
提交后转入列表界面，可查看作业列表：

点击相应作业的作业名可进入作业详情页



选择“数据”标签页可对当前作业的数据进行管理，查看等操作





从上面的对话页面中，点击左上角“《”可回到列表界面，选中作业后可对作业进行管理

3. 命令行方式提交管理作业

当前集群部署的是 slurm 作业管理系统，当前有一个队列

1) 查询节点状态 sinfo

[public@hpcmgt ~]$ sinfo

```
[root@hpcmgt ~]# sinfo
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
cpu*         up   infinite      1    mix hpccpu03
cpu*         up   infinite      5  alloc hpccpu[01-02,04-06]
gpu          up   infinite      2   drng hpcgpu[06,08]
gpu          up   infinite      7    mix hpcgpu[07,09-11,13-15]
gpu          up   infinite      3  alloc hpcgpu[01-02,12]
gpu          up   infinite      1   idle hpcgpu03
```

以上表示：

alloc：计算资源全分配出去

drng：节点失去活力有问题

mix：部分资源占用，部分空闲

idle：整个节点空闲

2) 查询节点使用 pestat

```
[root@hpcmgt ~]# pestat
Hostname       Partition     Node Num_CPU  CPUload  Memsize  Freemem                 Joblist
                            State Use/Tot              (MB)     (MB)                 JobId User ...
hpccpu01            cpu*     idle   0  52    0.00   256000   252807
hpccpu02            cpu*     idle   0  52    0.00   256000   252847
hpccpu03            cpu*     idle   0  52    0.00   256000   252853
hpccpu04            cpu*     idle   0  52    0.00   256000   252803
hpccpu05            cpu*     idle   0  52    0.00   256000   252798
hpccpu06            cpu*     idle   0  52    0.00   256000   252824
hpcgpu01             gpu     idle   0  48    5.45*  384000   338740
hpcgpu04             gpu     idle   0  48    0.01   512000   510441
hpcgpu05             gpu     idle   0  48    0.00   512000   510439
[root@hpcmgt ~]#
```

3) 编辑一个作业脚本（单节点单核，串行）

```
[cadmin@hpcmgt ~]$ cat > job1.sh <<EOF
#!/bin/bash
#SBATCH --job-name Job1-test

sleep 60
hostname
EOF
```

4) 提交作业 sbatch

```
[cadmin@hpcmgt ~]$ sbatch job1.sh
Submitted batch job 72
```

提交成功后返回作业 ID 72

5) 查看作业队列状态

```
[cadmin@hpcmgt ~]$ squeue
             JOBID  PARTITION     NAME     USER  ST       TIME  NODES NODELIST(REASON)
                72    compute Job1-test   public   R       0:13      1 hpccpu01
```

以上信息表明 JOBID 为 72，作业名为 Job1-test 的作业正在运行，已运行时间 13 秒，占用节点数 1，正在节点 hpccpu01 上运行

6) 作业完成后查看输出

```
[public@m1 ~]$ ls
docs    job1.sh    slurm-72.out
[cadmin@hpcmgt ~]$ cat slurm-72.out
hpccpu01
[cadmin@hpcmgt ~]$
```

以上信息表明作业执行完成后产生了一个文件名为“slurm-72.out”的文件。

7) 一个 Python 作业的 Demo

```
#!/bin/bash
#SBATCH --nodes=1       # comments allowed
#SBATCH --ntasks=1
#SBATCH --time=00:10:00
#SBATCH --mem=500M
#SBATCH --output=python_job_slurm.out

# Pass your Python script to the Anaconda2 python intepreter for execution
python vectorization.py
```

8) 作业组 Demo

如果当前目录下有多个带序号的脚本要执行，如：test-1.sh test-2.sh test-3.sh；或者多个目录下都有一相

同名字的脚本要处理相应目录下的数据，如： 05/d.sh 06/d.sh 07/d.sh 08/d.sh 09/d.sh 10/d.sh

11/d.sh 。这

种情况下，每个任务都是串行的，实际上综合起来是并行的，而且各脚本之间及数据的处理也具备无关性，平时我们

可能会一个个去执行这些脚本，也可能使用后台 nohup 方式一起执行这些脚本，其实我们可以用 Slurm 将其合并成一个

作业组提交到各计算节点去执行，作业组提交时，参数 --array=X-Y, X,Y 参数介于 0-1000 之间，作业脚本如下：

```
#!/bin/bash
#SBATCH --ntasks=1
#SBATCH --job-name=test-array-script
#SBATCH --array=1-3

. test-${SLURM_ARRAY_TASK_ID}
```

```
#!/bin/bash
#SBATCH --ntasks=1
#SBATCH --job-name=test-array

#SBATCH --array=01-11

. ${SLURM_ARRAY_TASK_ID}/sh
```

另一个作业组的示例，如有 201601 201602 .. 201612 的数据目录，有个串行脚本 process.sh 可以处理这些目录

中的数据，一次可以处理一个目录，常规处理方式如下：

[cadmin@hpcmgt ~] $ ./process.sh 201601

我们可以使用循环来处理所有目录，如 for i in {201601..201612} ; do ./process.sh $i ; done ；但这种

方式还是一个目录一个目录地处理，当然还有另外的方式，即处理一个目录时把进程放入后台，让其并行处理。我们

使用 Slurm 作业调度系统可以用如下方式来处理

```
#!/bin/bash
#SBATCH --ntasks=1
#SBATCH --job-name=processDir

#SBATCH --array=1-12

Dir=`expr 201600 + ${SLURM_ARRAY_TASK_ID}`
./process.sh ${Dir}
```

以上作业脚本中的${SLURM_ARRAY_TASK_ID}即作业组（数组）下标，然后麻烦的为对应数据目录，sbatch 提交这个

作业即可并行处理所有目录中的数据。注意 --array= 的写法，当然，作业脚本中不写，在执行 sbatch 命令提交作业时

作为参数写上效果也一样。

9) 一个 MPI 并行作业的 Demo

```
#!/bin/bash
```

```
#SBATCH --nodes=3
#SBATCH --tasks-per-node=8        # 8 MPI processes per node
#SBATCH --time=7-00:00:00
#SBATCH --mem=4G        # 4 GB RAM per node
#SBATCH --output=mpi_job_slurm.log
echo $SLURM_JOB_NODELIST


mpirun ./test
```

有的资料上说可以使用 srun 来代替 mpirun，经实测，有个先决条件，就是要提前把 mpd 启动好，这样就不如使用

mpirun 命令方便了。

10) 一个多线程并行作业的 Demo

```
#!/bin/bash
#SBATCH --mail-user=myemail@vanderbilt.edu
#SBATCH --mail-type=ALL
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4 # 4 threads per task
#SBATCH --time=02:00:00 # two hours
#SBATCH --mem=4G
#SBATCH --output=multithread.out
#SBATCH --job-name=multithreaded_example


# Run multi-threaded application
./hello
```

11) 取消一个作业 scancel

如果提交了一个作业并且这个作业无论是否已经在运行状态，这时候突然觉得因为什么原因要停止这个作业，可以使用

scancel 命令来停止这个作业，执行 scancel <job_id> 即可，也可以执行 scancel --help 获取帮助。

12) 提交作业到某个 partition

sbatch -p <patition_name> demo.sh

13) 查询单个作业详情：

scontrol show job <jobid>

例如 scontrol show job 100027

即可查询改作业当前资源申请量、所在目录、输出文件、排队或运行信息等内容：

```
[root@hpcmgt ~]# scontrol show job 192616
JobId=192616 JobName=Merge_S2_b1_Tau0.5_rho0.5_N2000_p5000_m50
   UserId=v530202707(316056) GroupId=domain users(513) MCS_label=N/A
   Priority=4294894788 Nice=0 Account=v530202707 QOS=normal
   JobState=PENDING Reason=DependencyNeverSatisfied Dependency=afterok:192615_*(failed)
   Requeue=1 Restarts=0 BatchFlag=1 Reboot=0 ExitCode=0:0
   RunTime=00:00:00 TimeLimit=00:30:00 TimeMin=N/A
   SubmitTime=2026-02-28T11:37:35 EligibleTime=Unknown
   AccrueTime=Unknown
   StartTime=Unknown EndTime=Unknown Deadline=N/A
   SuspendTime=None SecsPreSuspend=0 LastSchedEval=2026-02-28T11:37:35 Scheduler=Main
   Partition=cpu AllocNode:Sid=hpccpu03:647170
   ReqNodeList=(null) ExcNodeList=(null)
   NodeList=
   NumNodes=1 NumCPUs=1 NumTasks=1 CPUs/Task=1 ReqB:S:C:T=0:0:*:*
   TRES=cpu=1,mem=8G,node=1,billing=1
   Socks/Node=* NtasksPerN:B:S:C=0:0:*:* CoreSpec=*
   MinCPUsNode=1 MinMemoryNode=8G MinTmpDiskNode=0
   Features=(null) DelayBoot=00:00:00
   OverSubscribe=OK Contiguous=0 Licenses=(null) Network=(null)
   Command=(null)
   WorkDir=/home/student/v530202707
   StdErr=/home/student/v530202707/logs/S2_b1_Tau0.5_rho0.5_N2000_p5000_m50/merge.err
   StdIn=/dev/null
   StdOut=/home/student/v530202707/logs/S2_b1_Tau0.5_rho0.5_N2000_p5000_m50/merge.out
   Power=
```